Statistical Code Analysis Report

Backend

Backend static code analysis was done with four tools: SonarQube, CheckStyle, SpotBugs, and PMD.

SonarQube

SonarQube analysis was done in Multi-Quality Rule (MQR) mode, which aims to represent the impact of issues on all software qualities. It means that one issue may belong to multiple categories and inflate the issue count. The SonarQube analysis overview is shown in Figure 1. In total, 161 issues were found, which are elaborated on in Table 1.

It is worth noting that in addition to code logic, the coverage percentage also counts test folder into the calculation, which lowers the total coverage. In actuality, the backend code logic has a coverage of 57%.

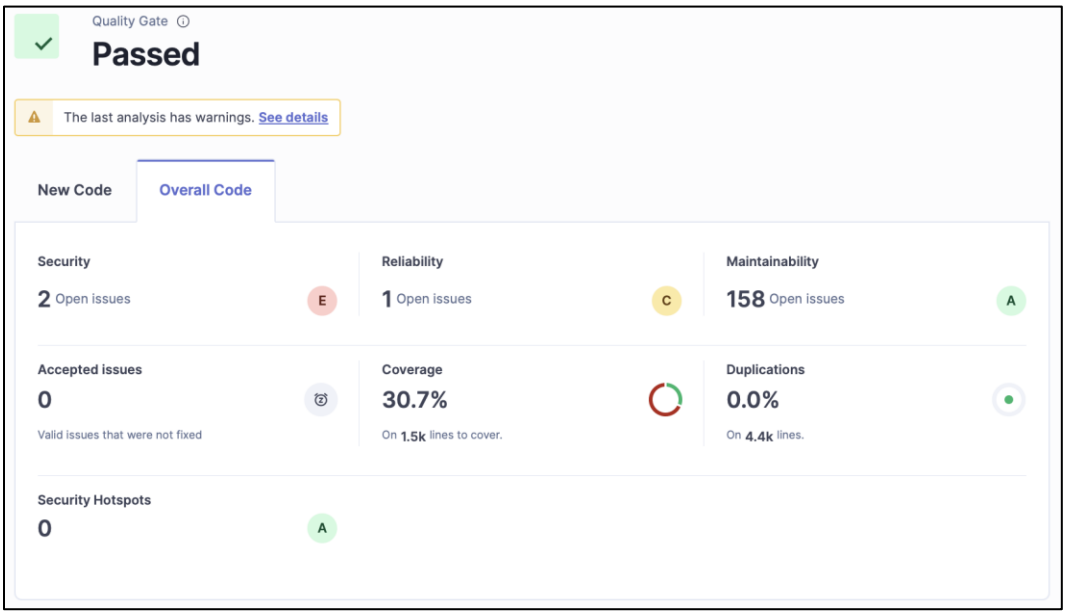

Figure 1. Backend SonarQube analysis results overview.

Table 1. Backend static code analysis by SonarScanner and SonarQube.

Software Quality	**Issues (count)**	**Effort (to solve)**	**Severity**	**Issue type(s)**
Security	2	2h	Blocker (2)	Responsibility (2)
Reliability	1	15mins	Medium (1)	Intentionality (2)
Maintainability	158	2d 6h	High (81) Medium (19)	Intentionality (52) Consistency (29)

			Low (57) Info (1)	Adaptability (77)

Code metrics are presented in table 2. Cyclomatic complexity is high as it adds together all the complexities present in individual files. Lines of code per method are acceptable, and there is no duplicate or unreachable code.

Table 2. Code metrics in SonarQube.

Cyclomatic Complexity	396
Lines of code per method (lines of code / functions) Lines of code Functions	10.621 3197 301
Duplicate or unreachable code	0

CheckStyle

Static analysis was conducted using CheckStyle with Sun Checks configuration. The analysis covered 51 files, producing a total of 1,099 issues. Most common issue categories were documentation, formatting and naming. Issues by category are listed in Figure 2, and by package are listed in Figure 3.

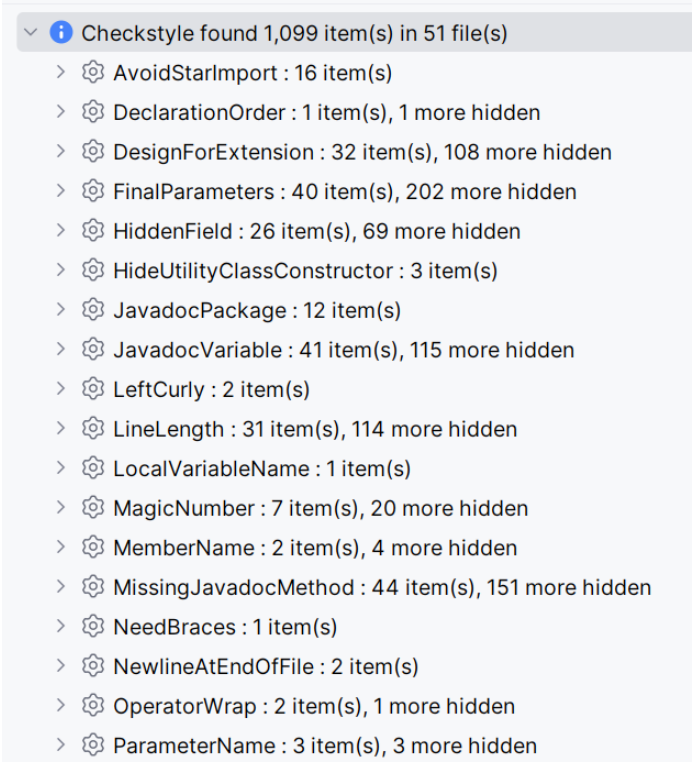

Figure 2. Issues by category.

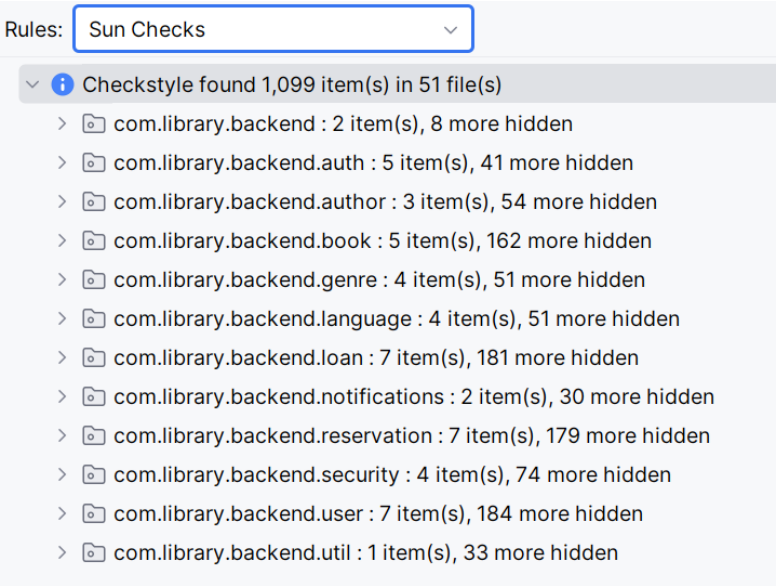

Figure 3. Issues by packages.

The analysis revealed a high number of violations, primarily related to code readability, documentation, and coding standards rather than critical functional errors. Missing documentation included violations with MissingJavadocMethod (44 + 151 hidden) and JavadocVariable (41 + 115 hidden) violations. This indicates that many methods and variables are not properly documented, which reduces maintainability and makes the code harder to understand. Examples of missing documentation are shown in Figure 4.

Figure 4. Violations in Book.java file.

There were also issues related to parameters and field usage. FinalParameters (40 + 202 hidden) suggest that many parameters are not declared as final, while HiddenField (26 + 69 hidden) indicates cases where local variables hide class fields.

Code style violations are also common, particularly LineLength (31 + 114 hidden), along with whitespace and operator formatting issues. For example, lines exceeding 80 characters reduce readability.

CheckStyle does not analyze test code located under the test source directory (e.g. src/test/java) as part of its standard configuration. As a result, unit tests are excluded from the static analysis scope, and any style or documentation issues within tests are not reported.

Spot Bugs

Static code analysis was performed using SpotBugs in the IntelliJ IDEA environment for purpose of identifying common coding issues based on known bug patterns. The analysis covered 54 classes, of which 29 bug items were found. Most of the identified bugs, shown in figure 5, originate from similar causes. The severity of bugs varies from medium to high, portrayed in figure 6.

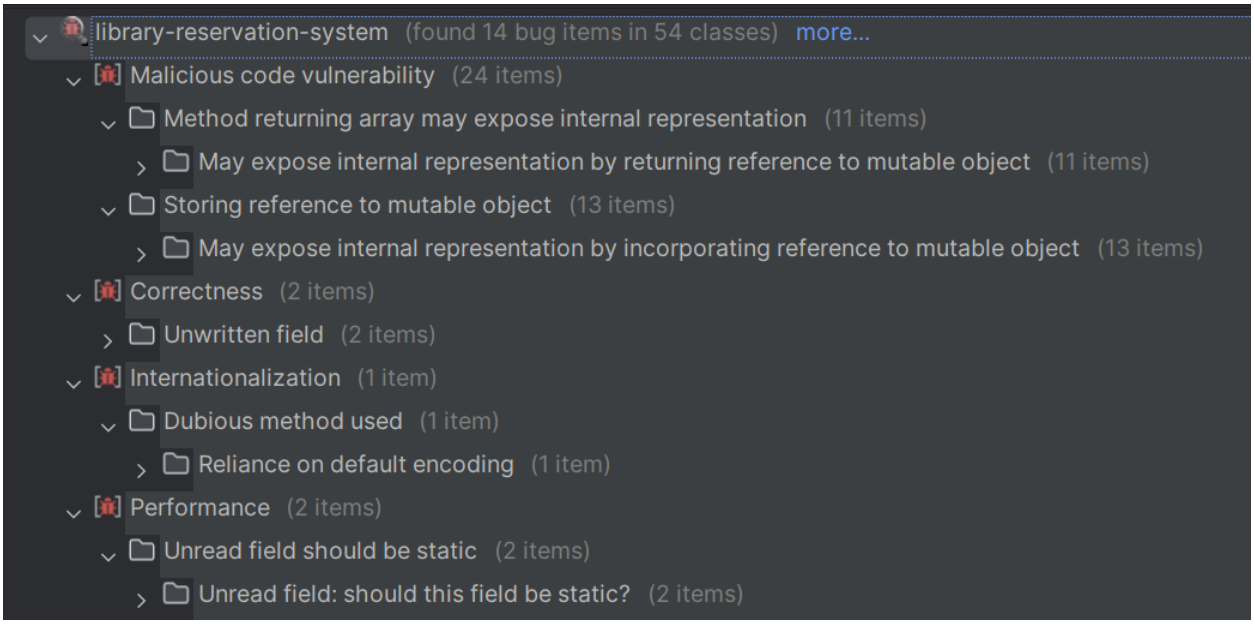

Figure 5: SpotBugs analysis grouped by bug category.

Over 83% of issues derive from exposure of the internal data of objects. This happens when mutable objects are either returned directly or stored without making a copy.

However, most of these links between entities have been made intentionally using ORM framework Hibernate. For example, the Book entity represents a single source of truth, and changes to it (e.g. updating the title) are expected to be visible in related loans.

Overall, 97% of the bugs were identified as medium confidence issues.

Out of the 29 bugs, the analysis identified one high confidence issue concerning internationalization. The absence of specified charset may lead to unreliable String-to-byte conversion, since the default encoding varies by system.

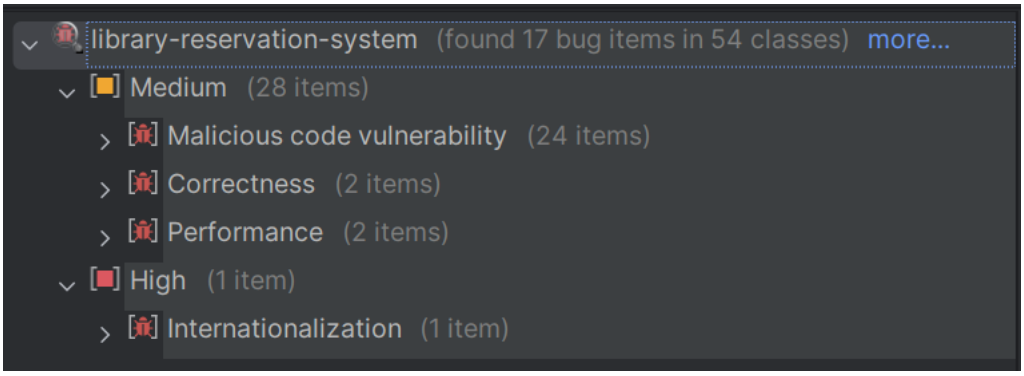

Figure 6 SpotBugs analysis grouped by priority.

PMD

Static code analysis was performed using PMD with 8 rule sets in IntelliJ IDEA. The analysis covered 70 files, producing a total of 1452 violations. The PMD results overview is shown in Figure 7, and a detailed breakdown by category is shown in Figure 8 and 9.

The largest category was codestyle with 694 violations, followed by documentation with 472 violations, and bestpractices with 150 violations. The remaining categories include errorprone 74, design 61 and multithreading 1.

Figure 7: PMD analysis results overview.

The majority of codestyle violations come from LocalVariableCouldBeFinal and MethodArgumentCouldBeFinal, which suggest that local variables and method parameters that are never reassigned should be declared as final. While these do not affect functionality, applying final improves code clarity that a value will not change. Documentation violations indicate that the majority of public methods, fields and constructors lack Javadoc comments. Most best practices violations relate to test code. JUnit5TestShouldBePackagePrivate recommends removing the public modifier from test classes, as Junit 5 does not require it. In design the higher severity issues were AvoidThrowingRawExceptionTypes, which indicates using generic RuntimeException instead of custom exception classes.

- PMD Results (1452 violations in 70 scanned files using 8 rule sets)
  - bestpractices (150 violations: 150)
    - JUnit5TestShouldBePackagePrivate (40 violations)
    - RelianceOnDefaultCharset (1 violation)
    - UnitTestAssertionsShouldIncludeMessage (54 violations)
    - UnitTestContainsTooManyAsserts (41 violations)
    - UnitTestShouldIncludeAssert (13 violations)
    - UnusedLocalVariable (1 violation)
  - codestyle (694 violations: 50 + 611 + 33)
    - FieldNamingConventions (10 violations)
    - FormalParameterNamingConventions (6 violations)
    - LocalVariableNamingConventions (5 violations)
    - MethodNamingConventions (29 violations)
    - AtLeastOneConstructor (24 violations)
    - CommentDefaultAccessModifier (22 violations)
    - ControlStatementBraces (1 violation)
    - LinguisticNaming (2 violations)
    - LocalVariableCouldBeFinal (286 violations)
    - LongVariable (13 violations)
    - MethodArgumentCouldBeFinal (239 violations)
    - OnlyOneReturn (5 violations)
    - ShortVariable (15 violations)
    - UnnecessaryCast (3 violations)
    - UseUnderscoresInNumericLiterals (1 violation)
    - BooleanGetMethodName (1 violation)
    - ShortClassName (3 violations)
    - UnnecessaryImport (27 violations)
    - UselessParentheses (2 violations)
  - design (61 violations: 5 + 56)
    - AvoidThrowingRawExceptionTypes (5 violations)

Figure 8: PMD detailed breakdown by category, highlighting best practices and code style violations.

- design (61 violations: 5 + 56)
  - AvoidThrowingRawExceptionTypes (5 violations)
  - AvoidCatchingGenericException (6 violations)
  - CyclomaticComplexity (1 violation)
  - DataClass (6 violations)
  - ExcessiveParameterList (1 violation)
  - FinalFieldCouldBeStatic (2 violations)
  - ImmutableField (6 violations)
  - LawOfDemeter (27 violations)
  - SignatureDeclareThrowsException (1 violation)
  - SimplifyBooleanReturns (1 violation)
  - UseObjectForClearerAPI (2 violations)
  - UseUtilityClass (3 violations)
- documentation (472 violations: 472)
  - CommentRequired (453 violations)
  - CommentSize (11 violations)
  - UncommentedEmptyConstructor (7 violations)
  - UncommentedEmptyMethodBody (1 violation)
- errorprone (74 violations: 74)
  - AvoidCatchingGenericException (6 violations)
  - AvoidDuplicateLiterals (44 violations)
  - AvoidFieldNameMatchingTypeName (2 violations)
  - AvoidLiteralsInIfCondition (1 violation)
  - AvoidUsingOctalValues (4 violations)
  - CollectionTypeMismatch (1 violation)
  - ReplaceJavaUtilDate (10 violations)
  - UseLocaleWithCaseConversions (6 violations)
- multithreading (1 violation: 1)
  - UseConcurrentHashMap (1 violation)

Figure 9: PMD detailed breakdown by category, highlighting design, documentation, error-prone and multithreading violations.

Recommendations:

- Solve issues starting from highest severity and moving down to lowest severity (Blocker -> High -> Medium -> Low -> Info).
- Increase test coverage to 80%.
- Improve documentation: add (Javadoc) comments to public methods and classes
- Use final parameter where applicable

- Fix field shadowing: avoid variable names that hide class fields
- Remove unused and redundant code
- Specify charset when necessary
- Rename localization fields in Book entity to camelCase to follow Java naming conventions

Frontend

Backend static code analysis was done with one tool: SonarQube.

SonarQube

Frontend SonarQube analysis was also done in Multi-Quality Rule (MQR) mode. The analysis overview is shown in Figure 2. In total, 268 issues were found, which are elaborated on in Table 3.

Frontend tests do exist, but they do not show up in this report and therefore the coverage is falsely at 0%.

Figure 10. Frontend SonarQube analysis results overview.

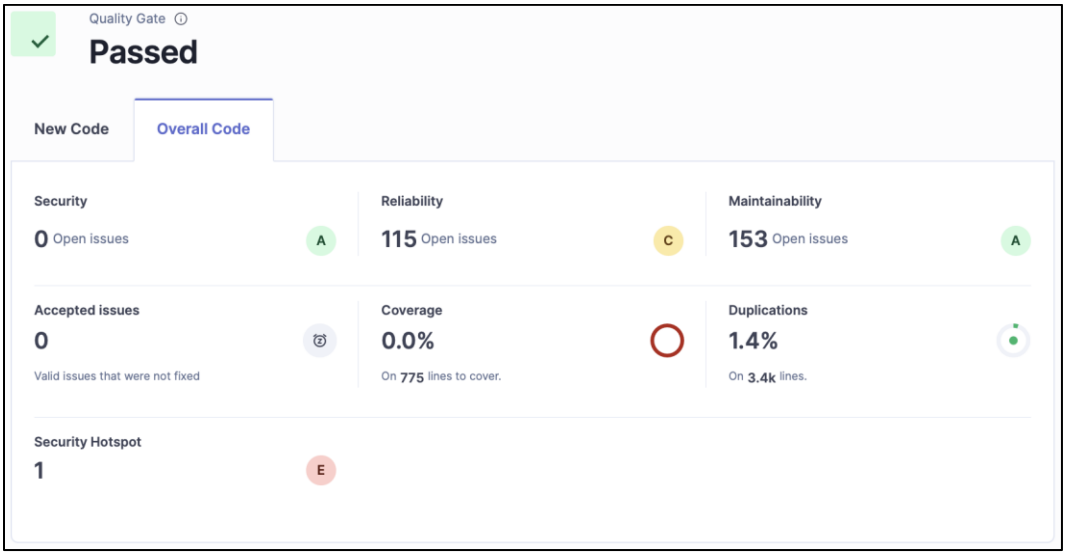

Table 3. Frontend static code analysis by SonarScanner and SonarQube.

Software Quality	Issues (count)	Effort (to solve)	Severity	Issue type(s)
Reliability	115	1d 2h	Medium (12) Low (103)	Consistency (106) Intentionality (9)
Maintainability	153	1d 4h	High (7) Medium (118)	Consistency (107) Intentionality (41) Adaptability (5)

			Low (25) Info (3)	

Code metrics are presented in table 4. Cyclomatic complexity is very high, lines of code per method are acceptable, and there were some duplicate lines of code.

Table 4. Code metrics in SonarQube.

Cyclomatic Complexity	459
Lines of code per method (lines of code / functions) Lines of code Functions	9.601 2861 298
Duplicate or unreachable code	46 (lines)

Recommendations:

- Refactor code to eliminate duplicate lines.
- Solve issues starting from highest severity and moving down to lowest severity (Blocker -> High -> Medium -> Low -> Info).